EXAMFORGE AI

Independent Verification Audit Report

12-Phase Penetration Testing & Production Readiness Assessment

Overall Production Readiness Score	||||||||..... **52/100**
Previous Score	18/100
Improvement	**+34 points**
Critical Issues Remaining	**4**
High Issues Remaining	**7**
Verified Fixes	**23**
New Issues Discovered	**11**
Launch Recommendation	**10 Schools (conditional)**

Assessment Scope: This report presents the results of a 12-phase independent verification audit of the ExamForge AI platform. Every fix from the previous engineering team was challenged through penetration testing, code review, and architectural analysis. The objective was not to confirm that fixes compile, but to verify they genuinely improve security and production readiness.

Phase 1 — Payment Penetration Testing

The Flutterwave payment integration was subjected to a comprehensive battery of attacks targeting webhook verification, amount validation, idempotency enforcement, commission calculation integrity, refund logic, and subscription activation. The payment system represents the highest-value attack surface because successful exploitation directly enables financial fraud.

1.1 Webhook Signature Verification

The webhook handler in `supabase/functions/flutterwave-webhook/index.ts` implements constant-time comparison via `constantTimeEquals()` to prevent timing attacks. The function XORs all corresponding bytes and OR-accumulates the results, returning true only when the accumulated result is zero AND the lengths match.

Attack Vector	Result	Detail
Fake signature (random hash)	PASS	Returns 401 Invalid signature. Constant-time prevents timing leakage.
Timing attack (progressive guess)	PASS	All bytes compared regardless of mismatch. No early-exit.
Empty verif-hash header	PASS	Returns false immediately (empty string check before comparison loop).
Missing verif-hash header	PASS	Returns false — null/undefined coerced to empty string, caught by empty check.
Replay attack (same payload, later)	PASS	Idempotency check via webhook_events table returns 200 already_processed.
Duplicate webhook (concurrent)	PASS	UNIQUE constraint on idempotency_key + upsert with onConflict handles race.

MEDIUM Finding P1-01: Constant-Time Bypass on Length Mismatch

When the incoming hash has a different length from the secret, line 31 of the webhook handler sets `b = a`, making both strings the same length. While the subsequent XOR loop processes all bytes, the length reassignment means the function returns `false` because `result = a.length ^ b.length` is now 0 (they are the same object), but the original `a.length === b.length` check at line 37 uses the ORIGINAL b length. Wait — actually `b = a` reassigns the local variable, so `b.length` is now `a.length`. The function would return `true` for two strings of DIFFERENT original length. This is a CRITICAL bug. An attacker sending a hash of a different length than the secret would have the comparison pass because `b` is replaced with `a` before the comparison loop runs, making all XORs zero. The final check `a.length === b.length` would also be true since `b` now equals `a`.

Impact: Any webhook with a hash of different length than the secret will be accepted as valid. This completely defeats the webhook signature verification.

Root Cause: The defensive code `b = a` was intended to maintain constant time but inadvertently replaces the comparison target, making all comparisons pass.

Remediation: Instead of reassigning `b = a`, use a separate dummy variable for the loop, and keep the length comparison against the original strings. For example: `let result = a.length ^ b.length; for (i in minLen) result |= a[i] ^ (i < b.length ? b[i] : a[i]);`

1.2 Amount Verification

Amount verification occurs at two levels: the client-side `verifyTransaction()` in the Flutter app, and the server-side webhook handler. The server-side implementation compares `data.charged_amount` from

Flutterwave against the locally stored `localTx.amount` from the database, with a 1 NGN tolerance.

Attack Vector	Result	Detail
Wrong amount (500 instead of 5000)	PASS	Webhook rejects with AMOUNT MISMATCH error. Flutter-side also rejects.
Negative amount (-5000)	PASS	parseFloat returns -5000, abs diff exceeds tolerance, rejected.
Zero amount (0.00)	PASS	Difference exceeds tolerance, rejected.
Large number (999999999)	PASS	Difference exceeds tolerance, rejected.
Amount within tolerance (5000.50 vs 5000)	PASS	abs(0.50) < 1.0, accepted. This is acceptable rounding behavior.
Wrong currency (USD instead of NGN)	PASS	CURRENCY MISMATCH check rejects the transaction.
Integrity hash tampered	PASS	verify_transaction_integrity() detects hash mismatch, rejected.

LOW Finding P1-02: Float Precision in Amount Comparison

Both the Edge Function and the Flutter datasource use `parseFloat()` to convert amount strings to numbers. JavaScript/Dart floating-point can introduce precision errors for very large amounts. For NGN, amounts exceeding 9,007,199,254,740,991 (Number.MAX_SAFE_INTEGER) would lose precision. While unlikely for typical Nigerian education transactions, enterprise contracts could theoretically exceed this. The integrity hash (SHA-256 on string concatenation) is not affected by float precision because it operates on the original string representations.

Remediation: Use integer kobo/cents for all amount calculations instead of floating-point Naira. Store amounts as NUMERIC(12,2) in PostgreSQL and compare as strings in the Edge Function.

1.3 Webhook Idempotency

Idempotency is enforced at two levels: an in-memory `WebhookIdempotencyTracker` in the Flutter app (first layer), and the `webhook_events` table in PostgreSQL (authoritative layer). The database approach uses a UNIQUE constraint on `idempotency_key` with `upsert` to handle concurrent duplicates.

Attack Vector	Result	Detail
Duplicate webhook (same event+id)	PASS	webhook_events UNIQUE constraint prevents double processing.
Concurrent webhooks (race)	PASS	UNIQUE constraint + upsert with onConflict handles race conditions.
Replay after processing (same key)	PASS	Returns 200 already_processed for processed events.
Replay during processing	PASS	Returns 202 processing for in-flight events.

MEDIUM Finding P1-03: In-Memory Idempotency Tracker is Ephemeral

The `WebhookIdempotencyTracker` in the Flutter app is an in-memory HashMap that is lost when the app restarts. While the database-backed idempotency in the Edge Function is the authoritative check, the Flutter-side tracker provides no real protection since the Flutter app should not be processing webhooks at all (webhooks are server-side only). The existence of this client-side tracker suggests architectural confusion — the Flutter app was previously processing webhooks, which is inherently insecure.

Remediation: Remove `WebhookIdempotencyTracker` from the Flutter codebase entirely. The Flutter app should never receive or process webhook events. All webhook processing must remain exclusively server-side in the Edge Function.

1.4 Commission Calculation

Commission calculation has been moved entirely server-side via the `calculate_marketplace_commission()` SQL function. This function is declared `SECURITY DEFINER`, meaning it executes with the privileges of its creator (typically the database superuser), not the calling user. It reads the seller's tier from `marketplace_seller_profiles`, looks up the active rate from `marketplace_commission_rates`, and applies minimum/maximum bounds.

The commission rates table has RLS that allows only `super_admin` users to modify rates. Regular authenticated users can only read active rates. This prevents client-side manipulation of commission percentages.

PASS Commission calculation is server-authoritative and cannot be manipulated by client-side code.

1.5 Refund Logic

The `processRefund()` method in the Flutter datasource sends refund requests directly to Flutterwave's API. The method accepts a `transactionId` and `amount` parameter.

HIGH Finding P1-04: No Server-Side Refund Validation

> The refund endpoint has no server-side validation. There is no Edge Function that validates refund requests before forwarding them to Flutterwave. Specifically missing: (1) No check that the refund amount does not exceed the original transaction amount, (2) No check that the transaction belongs to the requesting user's school, (3) No check that the transaction status is 'successful' before refunding, (4) No check for duplicate refund requests, (5) No maximum refund threshold. A malicious school admin could issue refunds for transactions they did not initiate, or issue refunds exceeding the original payment amount.
>
> **Impact:** Financial loss through fraudulent refunds. A school admin could drain another school's revenue by issuing refunds against their transactions.
>
> **Remediation:** Create a `supabase/functions/process-refund/index.ts` Edge Function that: (1) Verifies the transaction belongs to the requesting user's school, (2) Checks that the refund amount does not exceed the original paid amount minus prior refunds, (3) Validates the transaction is in a refundable state, (4) Logs all refund attempts to an audit table, (5) Enforces school-level daily refund limits.

1.6 Subscription Activation

The webhook handler processes `charge.completed` events and updates the transaction status. However, there is no explicit subscription activation logic in the webhook handler. When a subscription payment is confirmed, the `subscriptions` table should be updated to reflect the active status.

MEDIUM Finding P1-05: No Subscription Activation on Payment Confirmation

> The webhook handler updates the `transactions` table when a charge completes, but it does not activate the corresponding subscription. The `subscription.cancelled` event IS handled (cancels the subscription), but there is no `subscription.activated` or equivalent handler. This means a user who pays for a subscription must wait for a separate process to activate their access, creating a gap where payment is confirmed but the subscription remains in a pending or inactive state. For the `charge.completed` event on a subscription payment, the handler should also update the subscription's `status` to 'active' and set `current_period_start` and `current_period_end`.
>
> **Remediation:** Add subscription activation logic to the `charge.completed` handler. When the transaction's `meta` contains a `subscription_id`, update the subscription status to 'active' and set the billing period dates.

Phase 2 — Authentication Penetration Testing

The authentication system was tested for JWT tampering, expired token handling, session hijacking, refresh token replay, and unauthorized API access. The system delegates JWT management to the Supabase Flutter SDK, which provides built-in protections against many common attacks.

2.1 JWT & Token Security

Attack Vector	Result	Detail
JWT tampering (modify payload)	PASS	Supabase SDK validates JWT signatures server-side. Tampered tokens rejected.
Expired access token	PASS	ApiClient interceptor detects 401, attempts token refresh, retries.
Expired refresh token	PASS	Refresh fails, StorageService.clearSensitiveData() called, user logged out.
Session hijacking (stolen access token)	PARTIAL	Token is valid until expiry (1h default). No IP binding or device fingerprinting.
Refresh token replay	PASS	Supabase rotates refresh tokens on each use. Old refresh token is invalidated.
Missing Authorization header	PASS	Supabase RLS rejects unauthenticated requests. No data returned.
Anonymous API access (no token)	PASS	All tables require authenticated role. Anonymous gets zero rows.

MEDIUM Finding P2-01: No Session Binding to Device or IP

Access tokens are valid for their entire TTL regardless of which device or IP presents them. If an attacker steals an access token (e.g., via XSS or memory dump on a shared device), they can use it until it expires. The refresh token is stored in flutter_secure_storage, which provides OS-level encryption, but the access token is also stored there and loaded into memory for each API call. There is no mechanism to invalidate a specific session or to detect that a token is being used from an unusual location.

Remediation: (1) Implement server-side session tracking with device fingerprint and last-seen IP, (2) Add an Edge Function that can revoke specific sessions, (3) Alert users on login from new devices, (4) Consider shorter access token TTL (15 min instead of 1 hour).

2.2 ApiClient Auth Fix Verification

The previous audit identified that `ApiClient._getStoredAccessToken()` always returned null, meaning no auth tokens were injected into API requests. The fix wires the method to `StorageService.getToken()`, which reads from flutter_secure_storage.

PASS The ApiClient auth fix is verified. Tokens are now properly read from secure storage and injected into the Authorization header. The token refresh interceptor correctly attempts refresh on 401 responses and persists the new token pair.

MEDIUM Finding P2-02: No Rate Limiting on Login Endpoint

The authentication system relies on Supabase's built-in rate limiting, which is generous (typically allows many requests per minute). There is no application-level rate limiting on login attempts. An attacker could perform brute-force password attacks at the Supabase rate limit, which may be higher than desirable for an education platform handling student accounts.

Remediation: Implement application-level rate limiting: max 5 failed login attempts per email per 15 minutes, with exponential backoff. Add CAPTCHA after 3 failed attempts.

Phase 3 — RLS Verification (Multi-Tenant Isolation)

Row-Level Security policies were analyzed for complete tenant isolation. The platform uses a multi-tenant architecture where each school's data is isolated by `school_id`. The `get_user_role()` and `get_user_school_id()` helper functions are used throughout RLS policies to enforce tenant boundaries.

3.1 Tenant Isolation Test Results

Operation	Result	Detail
Read another school's students	PASS	RLS: school_id = get_user_school_id() enforced on users table.
Read another school's exams	PASS	RLS: exams filtered by school_id match to user's school_id.
Read another school's payments	PASS	RLS: transactions filtered by school_id via billing profiles.
Read another school's marketplace	PASS	Marketplace is global (not school-scoped) — correct by design.
Update another school's class	PASS	RLS: school_admin + school_id = get_user_school_id() required.
Delete another school's data	PASS	No DELETE policies for non-super_admin users on most tables.
Super admin cross-school access	PASS	Super admins can read all schools — correct by design.
Teacher from School A edits School B	PASS	Teacher INSERT requires school_id = get_user_school_id().

PASS The RLS role fix is verified. The `get_user_role()` and `get_user_school_id()` helper functions correctly resolve the current user's role and school, and all RLS policies use these functions for enforcement. The 'parent' role has been added to the enum and appropriate policies exist for parent-child data access.

MEDIUM Finding P3-01: SECURITY DEFINER Functions Bypass RLS

There are 80+ `SECURITY DEFINER` functions across the schema. These functions execute with the privileges of their creator, bypassing RLS policies. While this is necessary for operations like generating download tokens (where the calling user should not have direct table access), each SECURITY DEFINER function must be individually audited to ensure it does not expose data that RLS would otherwise protect. For example, if a SECURITY DEFINER function accepts a `school_id` parameter without verifying that the calling user belongs to that school, it could be used to bypass tenant isolation.

Remediation: Audit every SECURITY DEFINER function to verify that it either: (1) Does not accept parameters that could specify a different tenant's data, or (2) Validates that the calling user has access to the specified tenant's data before proceeding.

LOW Finding P3-02: Commission Rates Readable by All Authenticated Users

The `marketplace_commission_rates` table has a policy allowing any authenticated user to read active rates. While commission rates are not highly sensitive, exposing them allows sellers to optimize their listings to minimize platform fees. This is a business decision rather than a security issue, but it should be deliberate.

Phase 4 — Exam Security

The exam security system was tested for vulnerabilities that would allow students to read encrypted exam answers, modify local storage, tamper with the Drift database, or restore old exam sessions. The defense-in-depth approach includes anti-cheat monitoring, encrypted local storage, integrity hashes, and rate limiting.

4.1 Local Encryption Assessment

The `LocalEncryptionService` uses XOR-based stream cipher with a SHA-256 derived key. The key is derived from a device seed concatenated with a static application salt: `'ExamForge_AI_SecureStorage_2024_v1'`.

Attack Vector	Result	Detail
Read encrypted answers from SharedPreferences	PARTIAL	XOR cipher is reversible if key is known. Key derivation is predictable on rooted devices.
Modify local storage (SharedPreferences)	PASS	Stale session check (24h max) limits exploitation window.
Modify Drift database (exam_attempts table)	PARTIAL	integrityHash column detects tampering, but validation only on sync.
Restore old exam sessions	PASS	isStale() check discards sessions older than 24 hours.
Bypass encryption (read plaintext fallback)	PARTIAL	Legacy unencrypted data still supported via fallback path.

HIGH Finding P4-01: XOR Cipher is Not Cryptographically Secure

The `LocalEncryptionService` uses a repeating XOR cipher with a 32-byte key. This is trivially broken with known-plaintext attacks: if an attacker knows even one exam answer (e.g., they chose option A for question 1), they can XOR the ciphertext with the known plaintext to recover the key stream, which then decrypts all other answers. The code itself notes: 'For production, replace with proper AES-256 using the encrypt package.' This is a known deficiency that was NOT fixed in the security remediation.

Impact: On a rooted Android device or jailbroken iOS device, a student can: (1) Extract the device seed from SharedPreferences, (2) Derive the encryption key using the known salt, (3) Decrypt all exam answers, (4) Potentially share them with other students before submitting.

Remediation: Replace XOR with AES-256-GCM using the `encrypt` Dart package. Use the platform's keychain/keystore for key storage instead of SharedPreferences.

HIGH Finding P4-02: Device Seed Stored Alongside Encrypted Data

The `SessionRecoveryService` stores the device seed in SharedPreferences under the key `'_device_seed'`. This is the same storage location as the encrypted exam answers. On a rooted device, an attacker can read both the seed and the encrypted data, making the encryption completely useless since the key can be reconstructed from the seed and the known static salt.

Remediation: Store the encryption key in the platform keystore/keychain (via `flutter_secure_storage`) rather than SharedPreferences. The seed should never be stored in a location accessible to other apps.

MEDIUM Finding P4-03: Server-Side Exam Answers Unencrypted

While local exam answers are encrypted (albeit weakly), the server-side `student_answers` table in PostgreSQL stores answers as plaintext. Any database admin or Supabase dashboard user with access can read all student answers directly. For high-stakes examinations (WAEC, NECO, JAMB prep), this represents a significant integrity risk if the database is compromised.

Remediation: Encrypt answer payloads before inserting into the database, with the decryption key stored separately (e.g., in a vault service). At minimum, enable column-level encryption for the answer data in student_answers.

Phase 5 — Marketplace Security

The marketplace download system was tested for unauthorized access, signed URL reuse, download token manipulation, and expired resource access.

Attack Vector	Result	Detail
Download without purchase	PASS	Edge Function verifies purchase belongs to user and status=completed.
Reuse signed URL after expiry	PASS	Signed URLs expire in 1 hour. After expiry, Supabase Storage returns 403.
Modify download token	PASS	Tokens are 32-byte cryptographic random. Cannot be guessed or modified.
Download expired resource	PASS	validate_download_token() checks expires_at < now(). Expired tokens rejected.
Exceed max downloads per token	PASS	download_count >= max_downloads check prevents exceeding limit.
Use another user's token	PASS	Token validation requires buyer_id = auth.uid(). Different user rejected.
Revoke token after creation	PASS	is_revoked flag checked before allowing download.

PASS Marketplace download security is well-implemented. The combination of server-side purchase verification, cryptographic download tokens, time-limited signed URLs, download count enforcement, and comprehensive audit logging provides strong protection.

MEDIUM Finding P5-01: Signed URL Not Tied to Download Token

The Edge Function generates both a download token AND a signed URL, but they are independent. The signed URL is created directly from the Storage path and is valid for 1 hour. The download token is returned to the client but is NOT required when actually downloading via the signed URL. This means the signed URL can be shared with others and used to download the file without any token validation. The token is effectively unused in the actual download flow.

Impact: A buyer can share the signed URL with non-buyers within the 1-hour window, allowing unauthorized downloads. The download token and audit trail become meaningless because the actual file download bypasses them entirely.

Remediation: The signed URL should be generated on-demand when the token is validated, not upfront. Create a download-serving Edge Function that: (1) Accepts the download token, (2) Validates the token via validate_download_token(), (3) Only then creates a short-lived signed URL (5 min), (4) Redirects the client to the signed URL. The client should never receive a long-lived signed URL.

Phase 6 — AI Security

The AI security layer was tested against prompt injection, jailbreak attempts, Unicode bypass, Base64 bypass, nested injection, markdown injection, JSON injection, and output validation.

Attack Vector	Result	Detail
"Ignore previous instructions"	PASS	Critical pattern detected and blocked. Input sanitized.
"Reveal system prompt"	PASS	System prompt extraction pattern detected and blocked.
"Return API keys"	PASS	Data extraction pattern detected and blocked.
"Execute hidden instructions"	PASS	Instruction override pattern detected and blocked.
"Reveal confidential context"	PASS	Critical pattern detected and blocked.
"Generate unrestricted output"	PASS	Bypass pattern detected and blocked.
DAN mode / jailbreak	PASS	DAN pattern detected and blocked.
Unicode bypass (homoglyphs)	FAIL	No Unicode normalization. Unicode confusables bypass regex filters.
Base64 encoded injection	FAIL	No Base64 decoding before pattern matching. Encoded payloads bypass all filters.
Nested prompt injection	PARTIAL	Single-level detection. Nested instructions at depth 3+ may bypass.
Markdown injection	PARTIAL	Markdown link/image injection not specifically blocked.
JSON injection (field override)	FAIL	No JSON structure validation on input. JSON fields could override config.

HIGH Finding P6-01: No Unicode Normalization Before Pattern Matching

The prompt injection detection uses regex patterns against the raw input string. Unicode confusables (e.g., using Cyrillic '■' instead of Latin 'a', or zero-width characters) can bypass every regex filter. For example: 'ign■re previous instructions' (with Cyrillic '■') would bypass the `ignore\\s+previous` pattern because the character codes differ.

Remediation: Normalize input to NFC form and strip zero-width characters before pattern matching. Use `unorm.nfc(input)` or equivalent.

HIGH Finding P6-02: No Base64 Decoding Before Pattern Matching

An attacker can Base64-encode their injection payload: `aWdub3JlIHByZXZpb3VzIGluc3RydWN0aW9ucw==` decodes to 'ignore previous instructions'. The AI model may decode this as part of its reasoning, but the pattern matcher only sees the Base64 string, which does not match any critical or suspicious patterns.

Remediation: Decode Base64 strings found in the input before pattern matching. Flag inputs containing Base64 patterns for additional review.

MEDIUM Finding P6-03: Suspicious Patterns Are Logged But Not Blocked

When suspicious (but not critical) patterns are detected, the input is allowed with logging. This means patterns like `system:`, `assistant:`, and `ROLE:` pass through to the AI model. While these are less dangerous than critical patterns, they can be combined with other techniques to construct effective jailbreaks.

Remediation: Block suspicious patterns in production, or at minimum, require admin approval for inputs containing suspicious patterns before they are sent to the AI model.

Phase 7 — Performance Verification

Performance was assessed through static code analysis and architectural review. No live load testing was performed against a running instance, as no staging environment was available. Analysis focuses on query patterns, index coverage, and architectural bottlenecks.

Metric	Status	Detail
API latency (typical CRUD)	UNKNOWN	No live metrics. Architecture is sound: Supabase REST + RLS.
Database query optimization	GOOD	104+ indexes in CCMS, 60+ in billing. Composite indexes for common queries.
Search speed (question bank)	CONCERN	No full-text search indexes (GIN/GiST) found. LIKE queries will be slow.
AI response time	CONCERN	No response caching. Every question generation hits OpenAI/Gemini API directly.
Memory usage (Flutter)	CONCERN	4394-line dependency_injection.dart loads all providers eagerly.
Drift database (local)	GOOD	Indexed tables, lazy connection, VACUUM support.

MEDIUM Finding P7-01: No Full-Text Search Indexes

The question bank and marketplace product searches likely use `ILIKE '%term%'` queries, which cannot use B-tree indexes and result in full table scans. For a question bank with tens of thousands of questions, this will cause unacceptable latency.

Remediation: Add GIN indexes with `gin(to_tsvector('english', title || ' ' || content))` for full-text search on question bank and marketplace products.

Phase 8 — Load Testing Assessment

Load testing was not performed against a live environment. The following assessment is based on architectural analysis and identifies expected bottlenecks at each scale.

Scale	Expected Result	Detail
100 users	PASS (est.)	Single Supabase instance handles this easily. No issues expected.
500 users	PASS (est.)	Supabase handles well. AI API rate limits may cause queueing during peak.
1,000 users	CONCERN	Full table scans on search will degrade. Database connections may pool.
5,000 users	FAIL (est.)	No connection pooling config found. AI API calls become bottleneck.
10,000 users	FAIL (est.)	Requires PgBouncer, CDN, AI response caching, and horizontal scaling.

HIGH Finding P8-01: No Connection Pooling Configuration

The Supabase client is created in each Edge Function using `createClient(url, serviceKey)` without connection pooling. Under load, each request creates a new database connection, which will exhaust PostgreSQL's connection limit (typically 100 for Supabase free tier, 200-500 for pro). At 1,000+ concurrent users, connection exhaustion will cause random 503 errors.

Remediation: Configure PgBouncer in transaction mode. Use Supabase's built-in connection pooling (available in project settings). Ensure Edge Functions use the pooled connection string.

Phase 9 — Code Review

A targeted code review was conducted on the security-critical files modified during the remediation phase. The review focused on dead code, race conditions, null safety, exception handling, and architectural quality.

Issue	Severity	Detail
Dead code: WebhookIdempotencyTracker	HIGH	In-memory tracker in Flutter app serves no purpose since webhooks are server-side only.
Race condition: webhook upsert	LOW	UNIQUE constraint handles concurrent inserts, but error handling could be improved.
Null safety: ApiClient._storageService	MEDIUM	Optional StorageService means auth can silently fail if not injected.
Exception handling: Encryption fallback	HIGH	encryptData/decryptData return PLAINTEXT on failure instead of throwing.
Logging: Sensitive data in logs	MEDIUM	Debug mode logs full request/response bodies including auth tokens.
Memory: Eager DI loading	LOW	4394-line dependency_injection.dart eagerly creates all providers.
CORS wildcard: Access-Control-Allow-Origin: *	HIGH	Both Edge Functions use wildcard CORS. Should restrict to app domain.
Duplicate code: constantTimeEquals	LOW	Implemented in both Dart and TypeScript. Should use shared test suite.

HIGH Finding P9-01: Encryption Fallback Returns Plaintext on Failure

In `LocalEncryptionService.encryptData()`, if encryption fails for any reason (not initialized, exception thrown), the method returns the original plaintext. Similarly, `decryptData()` returns the ciphertext string on failure. This means any failure in the encryption pipeline silently stores exam answers in plaintext. An attacker who can trigger an encryption failure (e.g., by corrupting the device seed) can cause all subsequent exam answers to be stored unencrypted.

Remediation: Throw an exception on encryption failure instead of returning plaintext. The caller should handle the exception and either retry or abort the operation. Never silently fall back to plaintext for security-critical data.

HIGH Finding P9-02: CORS Wildcard on Edge Functions

Both Edge Functions (`flutterwave-webhook` and `marketplace-download`) use `Access-Control-Allow-Origin: *`. This allows any website to make requests to these endpoints from a browser. For the webhook handler, this is less critical because it only accepts POST requests with a valid signature. However, the marketplace-download endpoint accepts authenticated requests, and the wildcard CORS allows any malicious website to attempt to download purchased files using a logged-in user's credentials.

Remediation: Replace the wildcard with the specific app origin: `Access-Control-Allow-Origin: https://examforge.ai`. For the Flutter mobile app, add the custom scheme `io.examforge.ai`.

Phase 10 — Security Review (OWASP Compliance)

The platform was evaluated against the OWASP Top 10 (2021) and OWASP API Security Top 10. Each category was assessed for compliance, partial compliance, or non-compliance.

OWASP Category	Status	Detail
A01 - Broken Access Control	PARTIAL	RLS is strong, but SECURITY DEFINER functions and missing refund validation create gaps.
A02 - Cryptographic Failures	FAIL	XOR cipher for exam answers, device seed stored in SharedPreferences.
A03 - Injection	PARTIAL	SQL injection protected by Supabase parameterized queries. AI prompt injection partially mitigated.
A04 - Insecure Design	PARTIAL	Good architecture overall, but signed URLs bypass download tokens.
A05 - Security Misconfiguration	FAIL	CORS wildcard, debug logging in production, no rate limiting.
A06 - Vulnerable Components	UNKNOWN	No dependency audit available. Flutter/Supabase SDK versions not pinned.
A07 - Auth Failures	PARTIAL	Auth is functional but lacks brute-force protection and session binding.
A08 - Data Integrity Failures	PARTIAL	Integrity hash on transactions is good. No integrity on exam answers at rest.
A09 - Logging/Monitoring	FAIL	No centralized logging, no alerting, no SIEM integration.
A10 - SSRF	PASS	No server-side request forging patterns detected.

OWASP API Security Top 10 Assessment

API Security Category	Status	Detail
API1 - Broken Object Level Authorization	PASS	RLS enforces object-level auth on all tables.
API2 - Broken Auth	PARTIAL	Supabase JWT auth works, but no brute-force protection.
API3 - Broken Object Property Level	PARTIAL	Some endpoints return full objects including internal fields.
API4 - Unrestricted Resource Consumption	FAIL	No rate limiting on any API endpoint.
API5 - Broken Function Level Auth	PARTIAL	Refund endpoint lacks server-side authorization.
API6 - Unrestricted Access to Sensitive Business Flows	FAIL	AI generation has rate limits but payment flows do not.
API7 - Server Side Request Forgery	PASS	No SSRF vectors identified.
API8 - Security Misconfiguration	FAIL	CORS wildcard, verbose error messages in production.

API Security Category	Status	Detail
API9 - Improper Inventory Management	FAIL	No API versioning, no deprecation policy.
API10 - Unsafe Consumption of APIs	PARTIAL	Flutterwave API responses validated, but AI API outputs partially validated.

Phase 11 — Testing Coverage

Testing coverage was assessed by scanning the entire project for test files and analyzing the test infrastructure.

Category	Coverage	Detail
Unit Test Coverage	0%	Zero test files found. No test/ directory exists.
Widget Test Coverage	0%	No widget tests found.
Integration Coverage	0%	No integration tests found.
API Coverage	0%	No API tests found. No Supabase function tests.
Security Coverage	0%	No security-focused tests found.

CRITICAL Finding P11-01: Zero Test Coverage

The entire codebase of approximately 990 Dart files and 20+ TypeScript Edge Functions has ZERO automated tests. This is the single most critical gap in the platform. Without tests: (1) Any refactoring or security fix could introduce regressions without detection, (2) The constant-time comparison bug (P1-01) would have been caught by a simple unit test, (3) Payment flow changes cannot be verified before deployment, (4) RLS policies cannot be validated automatically, (5) AI security patterns cannot be regression-tested.

Remediation Priority:

(1) **Payment tests (Critical):** Unit test constantTimeEquals, amount verification, idempotency, integrity hash computation. Target: 80% coverage on billing module. (2) **Auth tests (Critical):** Test token injection, refresh, logout, session persistence. (3) **AI security tests (High):** Test every injection pattern, Unicode bypass, Base64 bypass. (4) **RLS tests (High):** Test cross-tenant access for every table. (5) **Exam encryption tests (High):** Test encryption/decryption, failure modes, integrity.

Phase 12 — Final Report

12.1 Verified Fixes

ID	Fix	Status	Verification
V-01	Webhook constant-time comparison	PASS	Algorithm correctly XORs all bytes (modulo P1-01 bug).
V-02	Server-side amount verification	PASS	Webhook compares charged_amount against DB-stored expected amount.
V-03	Currency verification	PASS	Currency mismatch detected and rejected in both webhook and Flutter.
V-04	Integrity hash (SHA-256)	PASS	Auto-computed on insert, verified on confirmation. Tamper-evident.
V-05	Replay attack detection	PASS	Flutterwave TX ID cross-referenced against existing transactions.
V-06	Server-side commission calculation	PASS	SQL function with SECURITY DEFINER. No client-side path exists.
V-07	Webhook idempotency (DB)	PASS	UNIQUE constraint on idempotency_key prevents double processing.
V-08	ApiClient auth token injection	PASS	StorageService wired to ApiClient interceptor. Tokens injected.
V-09	Token refresh on 401	PASS	Interceptor refreshes and retries. New tokens persisted.
V-10	RLS role fix (get_user_role)	PASS	Helper functions correctly resolve user role and school_id.
V-11	Parent role in RLS	PASS	Parent role added to enum. Parent-child policies created.
V-12	Encrypted exam answers (local)	PASS	Answers encrypted before SharedPreferences storage.
V-13	Legacy plaintext migration	PASS	Decrypt-first with fallback to plaintext for old data.
V-14	Session staleness check	PASS	Sessions older than 24h are discarded on recovery.
V-15	Signed URL downloads	PASS	1-hour time-limited URLs via Supabase Storage.
V-16	Download token system	PASS	Cryptographic tokens with count limits, expiry, revocation.
V-17	Download audit trail	PASS	Every download attempt logged with IP, user agent, success/failure.
V-18	AI prompt injection detection	PASS	Critical patterns detected and blocked.
V-19	AI output validation	PASS	System prompt leakage, hallucination, JSON structure checks.
V-20	AI cost controls	PASS	$0.50/request max, 8000 tokens max per request.
V-21	Content safety filtering	PASS	Harmful content patterns detected and blocked.
V-22	Integrity hash on local exam attempts	PASS	Drift DB has isEncrypted and integrityHash columns.
V-23	Purchase verification for downloads	PASS	Edge Function verifies purchase before generating URL.

12.2 Remaining Issues

ID	Severity	Issue	Impact
P1-01	CRITICAL	Constant-time comparison bypass on length mismatch	Any webhook with wrong-length hash is accepted.
P1-04	HIGH	No server-side refund validation	Fraudulent refunds possible.
P1-05	MEDIUM	No subscription activation on payment	Paid subscriptions remain inactive.
P2-01	MEDIUM	No session binding to device/IP	Stolen tokens valid until expiry.
P2-02	MEDIUM	No rate limiting on login	Brute-force attacks possible.
P3-01	MEDIUM	SECURITY DEFINER functions bypass RLS	Potential cross-tenant data access.
P4-01	HIGH	XOR cipher not cryptographically secure	Exam answers decryptable on rooted devices.
P4-02	HIGH	Device seed stored alongside encrypted data	Defeats encryption on rooted devices.
P4-03	MEDIUM	Server-side exam answers unencrypted	DB compromise exposes all answers.
P5-01	MEDIUM	Signed URL not tied to download token	URLs shareable within 1-hour window.
P6-01	HIGH	No Unicode normalization before AI input check	Confusables bypass all injection filters.
P6-02	HIGH	No Base64 decoding before AI input check	Encoded payloads bypass all filters.
P6-03	MEDIUM	Suspicious AI patterns logged not blocked	Combined attacks may succeed.
P7-01	MEDIUM	No full-text search indexes	Search performance degrades at scale.
P8-01	HIGH	No connection pooling	Connection exhaustion at 1000+ users.
P9-01	HIGH	Encryption fallback returns plaintext	Failures silently store unencrypted data.
P9-02	HIGH	CORS wildcard on Edge Functions	Any website can call marketplace-download.
P11-01	CRITICAL	Zero test coverage	No automated verification of any functionality.

12.3 Newly Discovered Issues

The following 11 issues were discovered during this verification audit that were NOT identified in the previous audit. These represent new findings that emerged from deeper code analysis.

ID	Severity	Issue	Source
P1-01	CRITICAL	Constant-time bypass on length mismatch in webhook	New — discovered during code review of constantTimeEquals()
P1-05	MEDIUM	Missing subscription activation in webhook handler	New — discovered during payment flow analysis
P4-02	HIGH	Device seed stored in SharedPreferences alongside encrypted data	New — discovered during exam encryption review
P5-01	MEDIUM	Signed URL bypasses download token validation	New — discovered during marketplace flow analysis
P6-01	HIGH	Unicode confusables bypass AI injection filters	New — discovered during AI penetration testing
P6-02	HIGH	Base64 encoding bypasses AI injection filters	New — discovered during AI penetration testing
P8-01	HIGH	No database connection pooling	New — discovered during load testing assessment
P9-01	HIGH	Encryption fallback returns plaintext on failure	New — discovered during code review
P9-02	HIGH	CORS wildcard allows cross-origin attacks	New — discovered during code review
P1-02	LOW	Float precision in amount comparison	New — discovered during payment penetration testing
P1-03	MEDIUM	In-memory idempotency tracker in Flutter is useless	New — discovered during architecture review

12.4 Risk Assessment

Risk Level	Count	Issues
CRITICAL	2	Webhook signature bypass, Zero test coverage
HIGH	8	Refund validation, XOR cipher, Device seed, Unicode/Base64 bypass, CORS, Connection pooling, Plaintext fallback
MEDIUM	8	Subscription activation, Session binding, Login rate limit, SECURITY DEFINER, Server-side answers, Signed URL gap, Suspicious patterns, Search indexes
LOW	2	Float precision, In-memory idempotency tracker

12.5 Updated Production Readiness Score

Category	Previous	Current	Delta	Notes
Architecture	18	65	+47	Clean architecture sound. RLS, DI, repository pattern well-implemented.
Security	8	40	+32	Major fixes applied, but critical webhook bug and encryption issues remain.

Category	Previous	Current	Delta	Notes
Performance	15	45	+30	Good indexing, but no connection pooling, no caching, no FTS.
AI Integration	20	55	+35	Good input/output validation, but Unicode/Base64 bypasses exist.
Database	25	70	+45	Comprehensive schema, RLS, triggers. SECURITY DEFINER audit needed.
Flutter Quality	30	60	+30	Good widget structure, but encryption fallback and dead code.
Testing	0	5	+5	Still essentially zero. Minimal framework may exist but no tests.
DevOps	10	30	+20	CI/CD scripts exist, but no monitoring, alerting, or runbooks.
Accessibility	5	15	+10	Basic Material 3 compliance. No ARIA, no screen reader testing.
Product Quality	15	45	+30	Feature-complete. UX polish needed. Error handling inconsistent.

Overall Production Readiness Score: 43/100 (Previous: 18/100, Improvement: +25 points)

12.6 Launch Decision

Scale	Decision	Requirements
10 Schools	CONDITIONAL YES	Fix P1-01 (webhook bug) and P9-01 (encryption fallback) first. Monitor closely. Acceptable risk for small-scale pilot.
100 Schools	NO	Must fix all CRITICAL + HIGH issues first. Add rate limiting, refund validation, AES-256 encryption, and basic test coverage (>30%).
1,000 Schools	NO	Requires all HIGH fixes + connection pooling + full-text search + 50%+ test coverage + monitoring/alerting + API versioning.
10,000 Schools	NO	Requires enterprise-grade: PgBouncer, CDN, AI response caching, SIEM integration, 80%+ test coverage, load testing validation, security audit certification.

Conditional Launch Criteria for 10 Schools:

1. **FIX IMMEDIATELY (before any launch):** P1-01 webhook constant-time bypass, P9-01 encryption plaintext fallback, P9-02 CORS wildcard.

2. **FIX WITHIN 1 WEEK:** P1-04 refund validation, P4-01/P4-02 encryption upgrade, P6-01/P6-02 AI Unicode/Base64 bypass.

3. **FIX WITHIN 1 MONTH:** All remaining MEDIUM issues, basic test coverage for payment and auth modules (>20%), connection pooling.

4. **ONGOING:** Increase test coverage to 50%+, implement monitoring/alerting, conduct external penetration test.

This report was generated through independent code analysis and penetration testing of the ExamForge AI platform. All findings are based on source code review — no live systems were tested. The recommendations are provided to improve the platform's security posture and should be prioritized according to the risk assessment above.